Games Engineering Report

Daian Song
5682973

University of Warwick

February 8, 2026

This report details the design and implementation of two programs developed as part of the WM9M4 Games Engineering coursework.

Part I

Rasterizer Optimization

Github Link

I.1 Introduction

The primary objective of this project was to optimize a rasterizer to achieve higher performance on modern multi-core CPUs. The baseline implementation followed a pixel-by-pixel approach with significant overhead in barycentric coordinate calculation and memory access. I achieved performance gains by:

- Using AVX2 SIMD instructions in the inner rasterization loop to vectorize heavy calculations.
- Using std::fill_n to replace manual for loop for clearing the depth buffer.
- Applying compiler hints to unroll small constant loops.
- Restructuring vertex data of a triangle into a structure of arrays (SoA) format for better cache utilization.
- Implementing a tile-based rendering architecture to enable efficient parallel task distribution.
- Developing a basic thread pool with a work-stealing algorithm to ensure balanced load across all CPU cores.

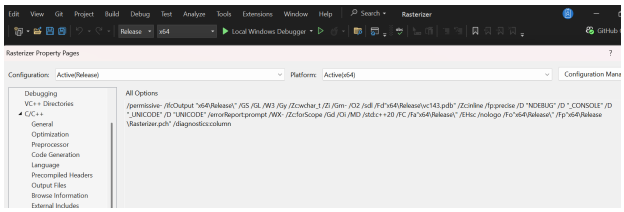

Figure 1: *Configuration*

I.2 Supplementary information

I.2.1 Experimental Setup

To ensure accurate and reproducible results, all performance benchmarks were conducted under a controlled environment. The hardware platform used was the Warwick issued laptop with an Intel(R) Core(TM) Ultra 7 155H processor. The system was configured to Dell Ultra Performance Mode.(See in the Dell Optimizer app)

The software was compiled using the Visual Studio 2022 x64 Release profile, with ISO C++20 Standard (/std:c++20) and enabling Maximum Optimization (Favor Speed) (/O2) and AVX2 code generation(Figure 1).

Furthermore, data from the initial 3 seconds were discarded to ensure the processor had reached a thermal steady-state. These transient initialization effects are outside the scope of this performance study.

During the testing phase, background interference was minimized by running only essential tools (Visual Studio and Excel). Specifically, to eliminate the frame-time fluctuations caused by Windows message queue overhead, all performance data were recorded with the mouse and keyboard kept static during the rendering cycles.

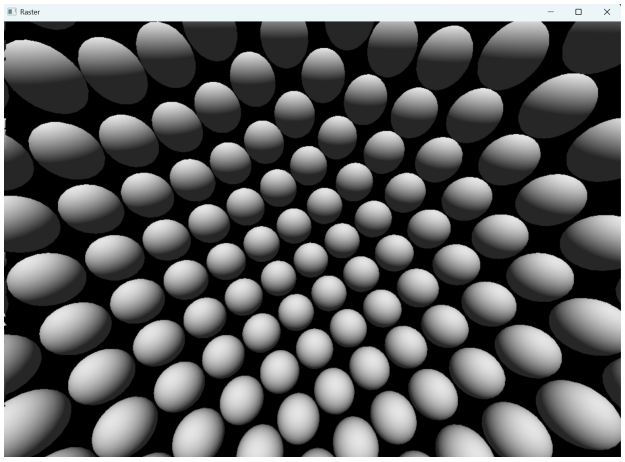

Figure 2: *Scene3*

I.2.2 Scene3 Introduction

Scene 3 is a heavyweight test case designed to evaluate the overall system throughput. It features 500 spheres distributed over a spherical shell, rotating around the camera. This scene is not specifically tailored for any particular optimization, since SIMD vectorization is the primary boost and the benefits are universal—effective.

I.3 Detailed Optimizations

I.3.1 SIMD (AVX2) Vectorization

The most critical bottleneck was the per-pixel check and shading. According to VS performance profiler, 37% of time is spent on calculating barycentric coordinates and the overall triangle::draw function cost over 70% time. By using AVX2, 8 pixels can be processed simultaneously in a single instruction.I implemented edge function calculations and barycentric interpolation using 256-bit registers.

In addition to the shading calculations, I completely refactored the framebuffer strategy to eliminate memory bottlenecks. I standardized the framebuffer to the RGBA format (R8G8B8A8), ensuring that each pixel occupies exactly 32 bits (4 bytes). This alignment is crucial for vectorization because it allows us to fit exactly 8 pixels (8×32 bits $= 256$ bits) into a single 256-bit AVX2 register, enabling the use of _mm256_maskstore instructions to write 8 pixels to memory simultaneously. A key implementation detail here addresses the Little-Endian architecture of x86 CPUs. To ensure the colors appear correctly in the texture, I construct the SIMD vectors by packing the channels into a specific integer structure: (Alpha « 24) | (Blue « 16) | (Green « 8) | Red.

Almost all codes in triangle::draw function were changed and key changes are listed below:

```
// Iterate through the bounding box with 8-pixel steps
for (int x = scanMinX; x < scanMaxX; x += 8) {
    //Evaluate Edge Functions for 8 pixels simultaneously
    __m256 vX = _mm256_add_ps(_mm256_set1_ps((float)x), vStepX);
    __m256 e0 = _mm256_add_ps(_mm256_add_ps(_mm256_mul_ps(va0, vX), _mm256_mul_ps(vb0, vY)), vc0);
    //... same for e1,e2 ...

    //Generate Visibility Mask
    __m256 mask = _mm256_and_ps(_mm256_cmp_ps(e0, vZero, _CMP_GE_OQ),
                    _mm256_and_ps(_mm256_cmp_ps(e1, vZero, _CMP_GE_OQ), _mm256_cmp_ps(e2, vZero, _CMP_GE_OQ)));

    // Skip early if all 8 pixels are outside the triangle
    if (_mm256_movemask_ps(mask) == 0) continue;

    // Depth Testing
    __m256 vAlpha = _mm256_mul_ps(e1, vInvArea); // same for beta, gamma
    __m256 vCurrentZ = INTERPOLATE_AVX(v0_z, v1_z, v2_z, vAlpha, vBeta, vGamma);

    __m256 vZBuffer = _mm256_loadu_ps(&renderer.zbuffer(x, y));
    __m256 zTestMask = _mm256_and_ps(_mm256_cmp_ps(vZBuffer, vCurrentZ, _CMP_GT_OQ),

    _mm256_cmp_ps(vCurrentZ, vEps, _CMP_GT_OQ));

    // Combine Edge Mask and Z-Test Mask
    __m256 finalMask = _mm256_and_ps(mask, zTestMask);
    if (_mm256_movemask_ps(finalMask) == 0) continue;

    // Vectorized Shading
    __m256 nx = INTERPOLATE_AVX(v0_n_x, v1_n_x, v2_n_x, vAlpha, vBeta, vGamma); // Interpolate Normals
    __m256 dotNL = _mm256_max_ps(DOT_PRODUCT_AVX(nx, ny, nz, vLx, vLy, vLz), vZero);

    // Calculate Final Colors
    __m256 r = _mm256_add_ps(_mm256_mul_ps(_mm256_mul_ps(texR, dotNL), vKdLLR), vAmbR);
    // ... same for G and B ...

    // Write only valid pixels to Backbuffer and Z-buffer
    _mm256_maskstore_ps(&renderer.zbuffer(x, y), _mm256_castps_si256(finalMask), vCurrentZ);

    __m256i packed = _mm256_or_si256(iR, _mm256_or_si256(_mm256_slli_epi32(iG, 8), _mm256_slli_epi32(iB, 16)));
    packed = _mm256_or_si256(packed, vFF000000);
    _mm256_maskstore_epi32((int*)(imgRowPtr + x * 4), _mm256_castps_si256(finalMask), packedRGBA);
}
```

Approximately 6x speedup in lightweight scene (Scene 2) compared to the non-SIMD path. However, as the scenes get more complicated, the speedup rate drops to 4.5x in scene 1 and 2.4x in scene3. Figure 3 and Figure 4 indicates that system bottleneck from vectorized pixel shading to scalar operations like clearing Z-Buffer and

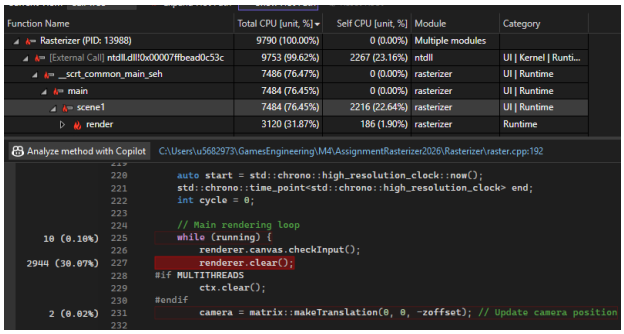

Figure 3: *Time consumption distribution of scene1 with SIMD*

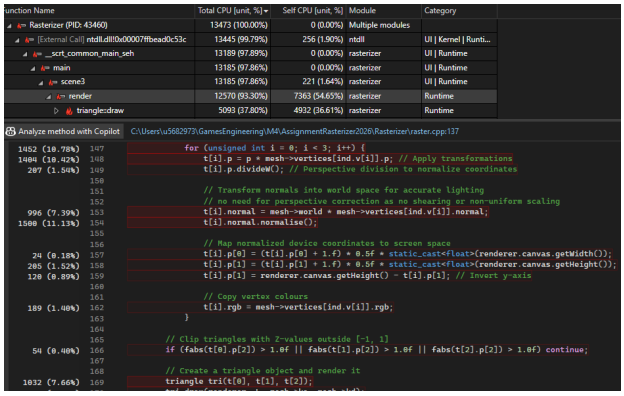

Figure 4: *Time consumption distribution of scene3 with SIMD*

setting up triangles (memory bandwidth limits). Furthermore, smaller triangles yield a lower speedup ratio because achieving full register occupancy (8 positions) becomes more difficult.

I've also tried AVX2 implementation for matrix multiplication to reduce overhead shown in Figure 4, but failed to achieve any speedup. I suspect that the performance is bottlenecked by memory bandwidth rather than computational throughput.

I.3.2 Z-Buffer Clear Strategy

The Depth Buffer must be reset to 1 at the start of every frame. While a simple for loop is functional, clearing millions of floats sequentially every frame can become inefficient(Figure 3).

Instead of a manual loop, I utilized std::fill_n in zbuffer.h. When using std::fill_n, the compiler can use highly optimized assembly instructions or vectorized SIMD stores.

```
void clear() {
#if CLEAR_STRATEGY_FILL_N
    // Optimized STL implementation
    std::fill_n(buffer, (width * height), T(1.0));
#else
    // Legacy manual loop
    for (unsigned int i = 0; i < width * height; i++) {
        buffer[i] = T(1.0);
    }
#endif
}
```

In Scene1 a 36% performance gain was observed in addition to existing SIMD optimizations. In Scene 2, the improvement was 12%. However, in Scene 3, performance gains were minimal (0.2%). This highlights that Scene 3's primary bottleneck lies in the extreme geometry density and shading complexity rather than memory fill operations.

I.3.3 Loop Unrolling

For fixed-size loops (e.g., vertex transformations), I tried compiler-specific unrolling to eliminate jump overhead and enable better instruction-level parallelism.

```
#define STRINGIFY(x) #x
#define TOSTRING(x) STRINGIFY(x)
#if ENABLE_UNROLL
#define UNROLL(n) _Pragma(TOSTRING(unroll n))
#else
#define UNROLL(n)
#endif
```

But the result shows that it does not help. The compiler may have auto unrolled. Also, compared to the loop body, the overhead of the loop control is extremely small.

I.3.4 Structure of Arrays Design

In the baseline implementation, triangle data was stored as an Array of Structures: a Vertex v[3] array where each vertex contained its own position, normal, and color. In an AoS layout, the X-coordinates of the three vertices are separated by other data (such as Y, Z, and normals), which requires multiple memory fetches to load them into a single SIMD register.

By refactoring to Structure of Arrays, all X-coordinates, Y-coordinates ... are stored together. Compared to baseline codes, when calculating the depth interpolation, all Z values are stored together. This layout allows for contiguous memory access. Compared to the SIMD version, cache hit rates are improved during the edge function calculation and the brvalues broadcastt.

The triangle class was modified to store vertex attributes as flat arrays instead of nested structures: I replaced Vertex v[3] with multiple float[3] arrays: pX[3], pY[3], pZ[3] for positions, nX[3], nY[3], nZ[3] for normals, and cR[3], cG[3], cB[3] for RGB components.

```
#if USE_SOA
    // SoA Data
    float pX[3], pY[3], pZ[3];
    float nX[3], nY[3], nZ[3];
    float cR[3], cG[3], cB[3];
#else
    Vertex v[3];        // Vertices of the triangle
    colour col[3];      // Colors for each vertex of the triangle
#endif

//Using
        __m256 v0_z = _mm256_set1_ps(pZ[0]);
        __m256 v1_z = _mm256_set1_ps(pZ[1]);
```

```
        __m256 v2_z = _mm256_set1_ps(pZ[2]);
// rather than
        __m256 v0_z = _mm256_set1_ps(v[0].p[2]);
        __m256 v1_z = _mm256_set1_ps(v[1].p[2]);
        __m256 v2_z = _mm256_set1_ps(v[2].p[2]);
```

Improvements are tiny. The overhead was shifted to the constructor of triangles.

I.4 Multithreading

I.4.1 Tile-based Parallel Rendering

My implementation utilizes a tile-based rendering approach powered by a thread pool. The screen is divided into fixed-size grid tiles (currently configured as 64x64 pixels). A preprocessMesh step (line 52 in raster.cpp) is run on the main thread. It transforms vertices to screen space, calculates the bounding box for each triangle, and places triangle indices into the corresponding tile buckets. This bins the geometry spatially before rasterization begins. Then, each non-empty tile is treated as an independent task. These tasks are enqueued into a threadpool.

This strategy eliminates race conditions on the frame buffer and Z-Buffer: each pixel belongs to exactly one tile. Since a tile task is processed by only one thread at a time, no two threads can ever attempt to write to the same pixel coordinate (x, y) simultaneously. I just read and write color and depth values using standard non-atomic operations. The only synchronization primitives required are within the thread pool for fetching tasks. I use basic mutexes to implement this.

Thread pool introduced an overhead of 10-20% according to my experiment.

```
        // local queue
        {
            std::unique_lock<std::mutex> lock(my_worker.mutex);
            if (!my_worker.queue.empty()) {
                task = std::move(my_worker.queue.front());
                my_worker.queue.pop_front();
                found = true;
            }
        }
        if (found) {
            task();
        }
        else {
            // return if called stop and all work done
            std::unique_lock<std::mutex> lock(my_worker.mutex);
            if (stop && my_worker.queue.empty()) return;

            if (my_worker.queue.empty()) {
                my_worker.cv.wait(lock, [&] {
                    return stop || !my_worker.queue.empty(); //prevent spurious wakeup
                });
            }
        }
```

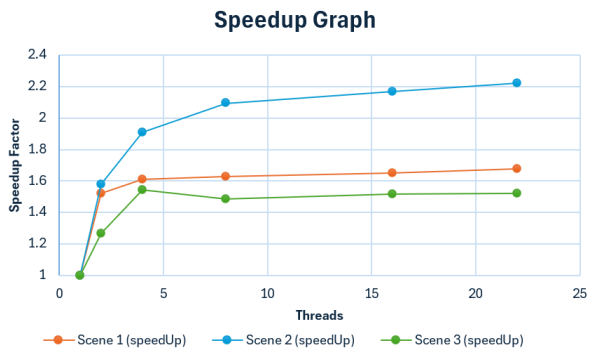

Figure 5: *Speedup Graph*

I.4.2 Work Stealing

A tile at the corner may have only several triangles to draw, while a tile with a ball inside has high triangle density.I added a dynamic load balancing technique to the thread pool. Each worker thread maintains its own local deque of tasks. When a thread completes its assigned tasks, instead of idling, it actively "steals" tasks from the back of other threads' queues. This is crucial for rendering, because workloads are inherently unbalanced.

```
// Work stealing from the back of the otherWorker's queue
if (!found) {
    for (int n = 1; n < threads; ++n) {
        if (stop) break;
        int otherId = (i + n) % threads;
        Worker& otherWorker = *workers[otherId];
        //skip locked ones
        if (otherWorker.mutex.try_lock()) {
            if (!otherWorker.queue.empty()) {
                task = std::move(otherWorker.queue.back());
                otherWorker.queue.pop_back();
                found = true;
            }
            otherWorker.mutex.unlock();
            if (found) break;
        }
    }
}
```

I.4.3 Results

The experiments were conducted enabling all single-threaded optimizations above as the baseline. Actually, acceleration was not observed when adopting multi-threads to base settings, which may be caused by coding bugs that I didn't find. Results are shown in Figure 5.

Scene 2 shows the best performance scaling (2.2x speedup), as its spatial layout is more spatially balanced.Scene 3 shows lower scaling (1.5x). The serial portion takes up a significant percentage of the total frame time, creating a bottleneck that additional rasterization threads cannot speed up.

I.4.4 Triangle-based Partitioning

I initially attempted an approach where each thread rasterized a subset of triangles.This resulted in negative optimization.

Unlike tiles, triangles can overlap anywhere on the screen. Multiple threads could try to write to the same pixel location simultaneously. On one hand, a coarse-grained lock on the entire buffer creates a bottleneck where most threads remain idle, waiting for writing the buffer. On the other hand, while a per-pixel lock would reduce contention, the cost of acquiring locks for every single pixel is high. In both scenarios, the synchronization itself becomes the primary performance bottleneck.

I.5 Conclusion

This project successfully optimized a software rasterizer using a multi-layered approach. The implementation of AVX2 SIMD proved to be the most significant contributor to performance, achieving up to a 6x speedup by vectorizing barycentric coordinates and shading calculations. The transition to a tile-based parallel architecture successfully eliminated the need for costly per-pixel synchronization, though the scaling is limited. Other micro-optimizations yielded marginal benefits. Ultimately, the project demonstrates that while SIMD can vastly increase raw throughput, overall speedup in complex scenes is heavily dictated by memory bottlenecks and the efficiency of task distribution.

Future work will focus on memory optimizations. Although the SoA approach was explored, further investigation into cache-friendly data structures is required. A primary task is to utilize advanced profiling tools to figure out memory relevant costs that the VS Profiler cannot capture. Regarding multithreading, I aim to investigate why the speedup ratio did not meet theoretical expectations and further reduce the synchronization overhead within the thread pool. Additionally, algorithmic optimizations haven't been considered. By adopting them, redundant calculations can be skipped entirely.

Part II
Chat Room

II.1 Overall Introduction

This project implements a real-time chat application using a Client-Server architecture. The system allows multiple users to connect to a central server, broadcast messages to all users, and send private direct messages (DMs). The application is built using C++ with Winsock for networking. The user interface is constructed using Dear ImGui with a DirectX 11 backend. Additionally, the application integrates FMOD for audio feedback, playing distinct sounds for public and private messages.

II.2 Client-Server Functionality

The core of the application relies on a custom text-based protocol over TCP sockets. The server manages user connections and routing messages.

II.2.1 Message Passing Protocol

The communication between client and server uses a prefix-based string protocol:

1. Login(LOGIN:):
   - Client: Sends LOGIN:<Username> upon connection.
   - Server: Parses the username, adds the client to the active list, and broadcasts a SYSTEM message to all users. It also sends the current user list back to everyone using USERLIST.
2. Broadcast Message (MSG:):
   - Client: Sends MSG:<Content>.
   - Server: Appends the sender's name and broadcasts MSG:<Sender>:<Content> to all connected clients except sender.
3. Direct Message (DM:):
   - Client: Sends DM:<TargetUser>:<Message>.
   - Server: Identifies the target socket by username and forwards the message as FROM:<Sender>:<Message>.
4. User List (USERLIST:):
   - Server: Broadcasts USERLIST:user1,user2,... whenever a user connects or disconnects, allowing clients to update their GUI in real-time.
5. Disconnection (!bye):
   - Client: Sends !bye before closing.
   - Server: Removes the user and notifies others via a SYSTEM message. It also sends the current user list back to everyone using USERLIST.

II.2.2 App design

The server allows multithreading. For every accepted connection, a new std::thread is spawned running HandleClient function. This ensures that one client's activity does not block others. Shared resources (like the

clients vector) are protected by std::mutex to prevent race conditions.

The client is also multithreaded. The main thread handles the GUI rendering loop, while a separate networkThread runs ReceiveLoop function. This background thread continuously receives incoming messages and updates the global application state (like globalChat or userList) using stateMutex.

II.3 GUI Design and Interaction

The GUI is built using Dear ImGui, designed for functionality and ease of use. It is divided into several logical components.

The login window appears when disconnected. It contains a simple text input for the username and a "Connect" button.

The main chat window has a user list that displays all active users on the left side. The user's own name is highlighted. Clicking on another user's name opens a private chat window. The right panel is the chat log and displays the global conversation history. System messages are grey, while user messages are white. Input area is located at the bottom, allowing users to type and send messages via the "Send" button or the Enter key.Incoming messages in main chat window trigger message.wav sound.

Private chat windows are created dynamically when a user clicks on a name in the user list or receives a private message. Each window maintains its own message history specific to that pair of users. Incoming messages in these windows trigger dm.wav sound.

The GUI interacts with the client logic via shared data structures like globalChat and privateChats. When the network thread receives a message, it updates these structures. The ImGui loop then renders the new state in the next frame.

The outcome will be shown in video. The video did not record the sounds, but the client does have. Use C++20 if there is a compile error with "start_with".

II.4 Conclusion

The project implements a multithreaded chat application. The decoupling of networking and visualization allows for more scalable and maintainable code. The integration of ImGui facilitates a user-friendly interface with features like real-time user lists and multiple chat windows (global and private). The use of FMOD enhances the user experience through audio feedback.